

Strona Portfela Fana – zarys kompletnej implementacji (Frontend + integracje)

Obszar Co trzeba zrobić Dlaczego / na co uważać

1 · Globalny stan & store Utwórz useWalletStore.ts z Zustand. Przechowuj: balance, transactions[], depositAddress, flagi isLoading, error, + asynchroniczne akcje:
ts fetchBalance() • fetchTransactions() • initiateCardDeposit() • requestWithdraw() Pozwala komponentom czytać / modyfikować dane portfela bez props-drillingu i reagować tylko na potrzebne zmiany. I**

4 · Modal doładowań Po CTA „Wpłać” → modal z dwoma ścieżkami:
1. Karta – frontend otwiera checkoutUrl zwrócony przez backend (np. hosted rampa Circle).
2. Krypto – wyświetl QR / adres depositAddress (USDC/Polygon). Oddzielenie „fiat-on-ramp” i „krypto-on-chain” upraszcza UX: użytkownik widzi tylko opcje, które zna i rozumie.

1. Fan odwiedza profil twórcy → wybiera kwotę, klika “TipIT!”

2. Frontend POST /api/tips → backend:

sprawdza saldo fana w Circle Wallet

tworzy transfer off-chain Wallet→Wallet (instant, \$0 gas)

zwraca success

3. UI wyświetla toast „Napiwek wysłany!”.

4. fetchBalance() & fetchTransactions() odświeżają dane – licznik USDC w panelu WalletOverview animuje się w górę / w dół.

Wszystkie powyższe kroki są możliwe do zrealizowania bez posiadania jakichkolwiek natywnych tokenów sieci przez fana, dzięki modelowi custodial wallets Circle i transferom księgowym wewnątrz systemu.

Check-list deweloperska (MVP)

[] Store Zustand + typy TS gotowe

[] Endpointy backend /api/wallet/* podłączone do Circle

[] Komponenty UI (Overview, DepositModal, HistoryList) ukończone

[] Walidacje kwot (≥ 0.01 USDC, max 2 miejsca po przecinku)

[] Skeletony/Toasty/Bannery błędów

[] Testy e2e (Cypress/Playwright) dla scenariuszy deposit / withdraw / tip

[] Podstawowe accessibility (tab-order, aria-label)

[] CI pipeline lint+type-check+unit-tests

Po odhaczeniu listy strona Portfela Fana jest gotowa do produkcyjnego MVP.